

Self Attention

C. V Jawahar

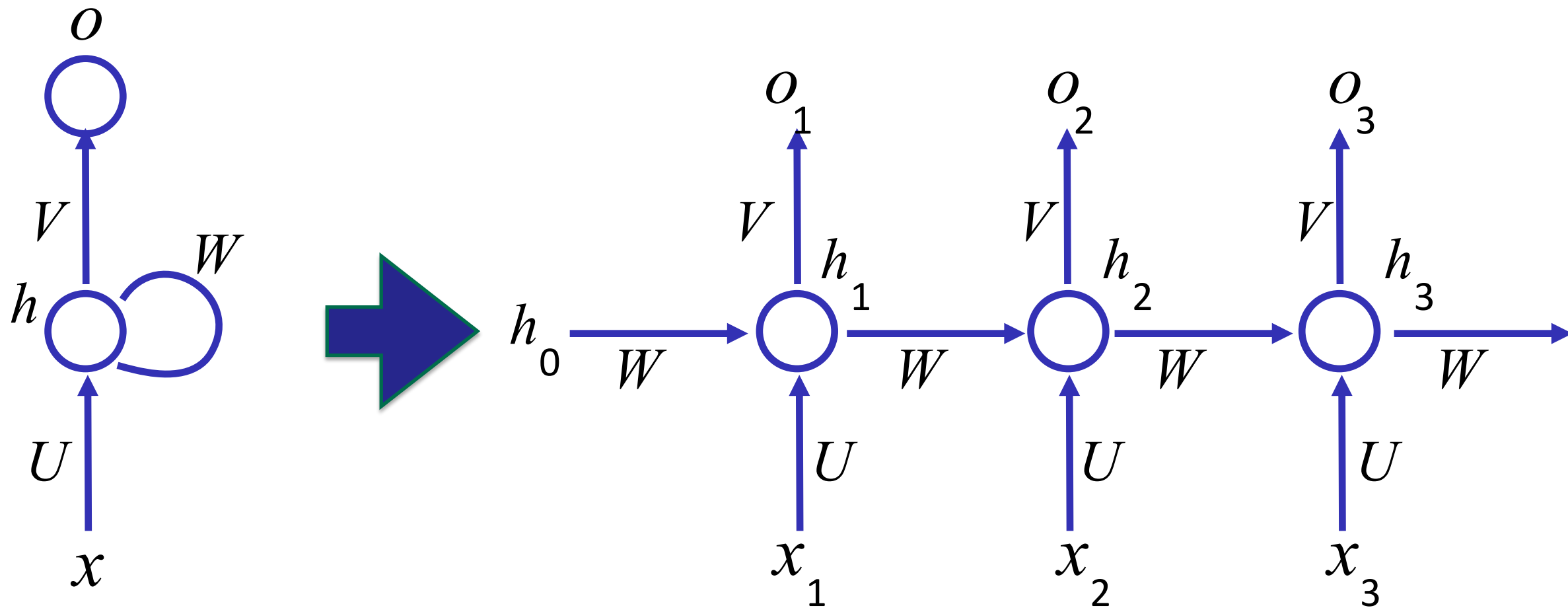
IIIT-Hyderabad

02 April 2026

Recap: CNNs

- 1D and 2D Convolutions
- Pad, Stride, Pool,
- Feature Maps
- Kernels, Windows,
- Weights, Parameters
- Design advancements
 - 1 X 1 Convolutions
 - Residual Connections
 - Deeper still smaller networks
- Pre-Train
- Fine Tune
- What do CNNs Learn?
- How do CNNs Learn?
- What type of inputs maximize activations?
- Different Performance Metrics/Concerns
- Different backbones

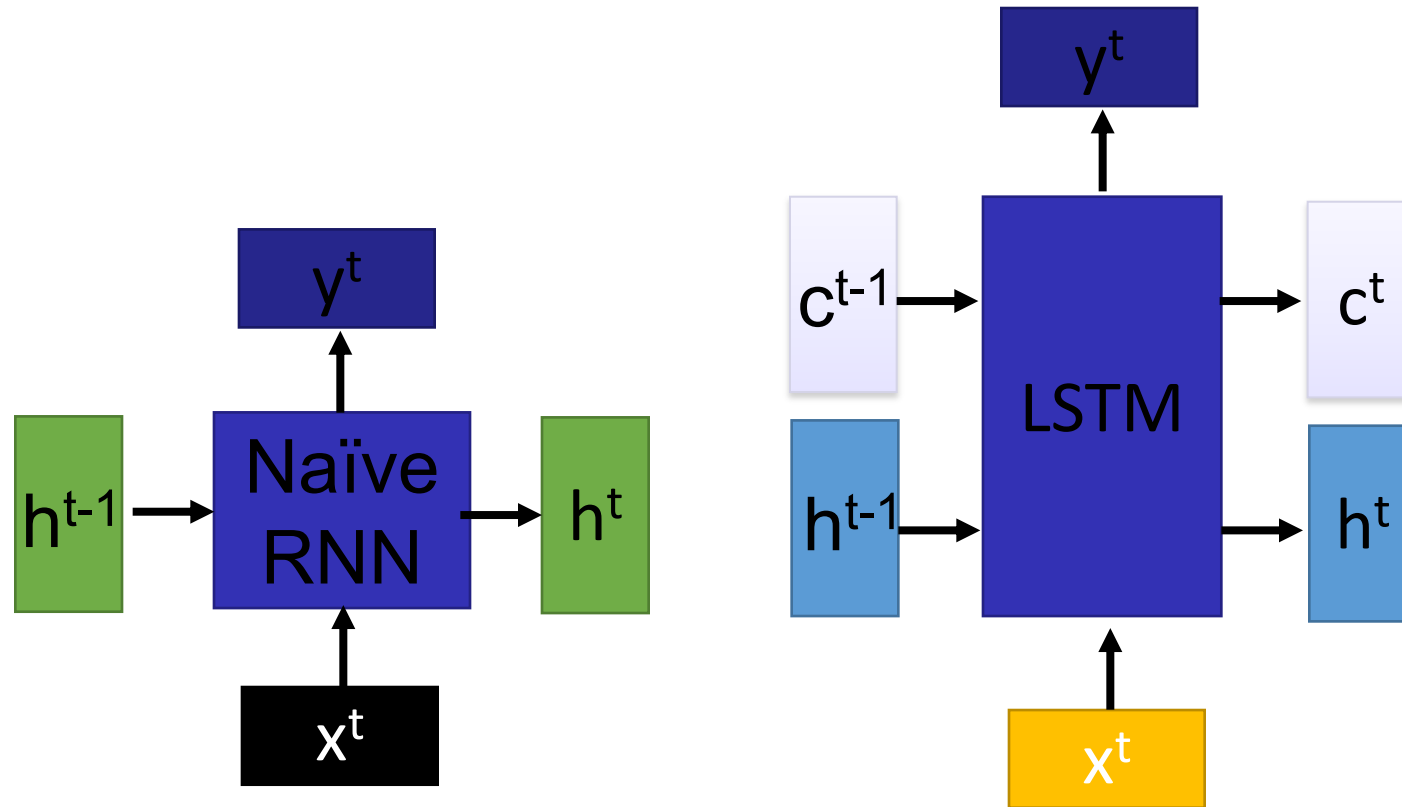
Recap: RNN



$$h_t = f(Ux_t + Wh_{t-1})$$

$$o_t = \text{softmax}_3(Vh_t)$$

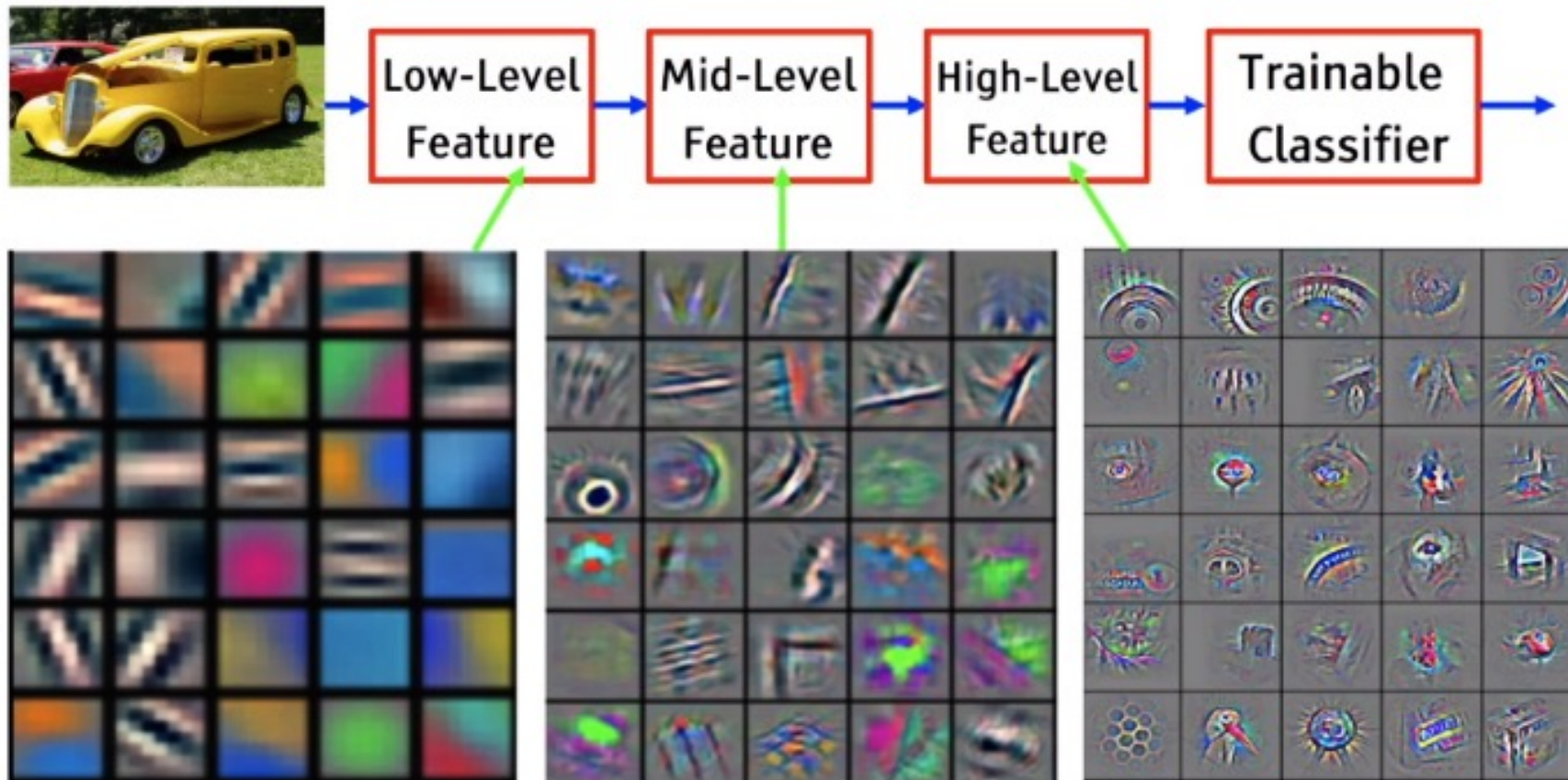
Naïve RNN vs LSTM



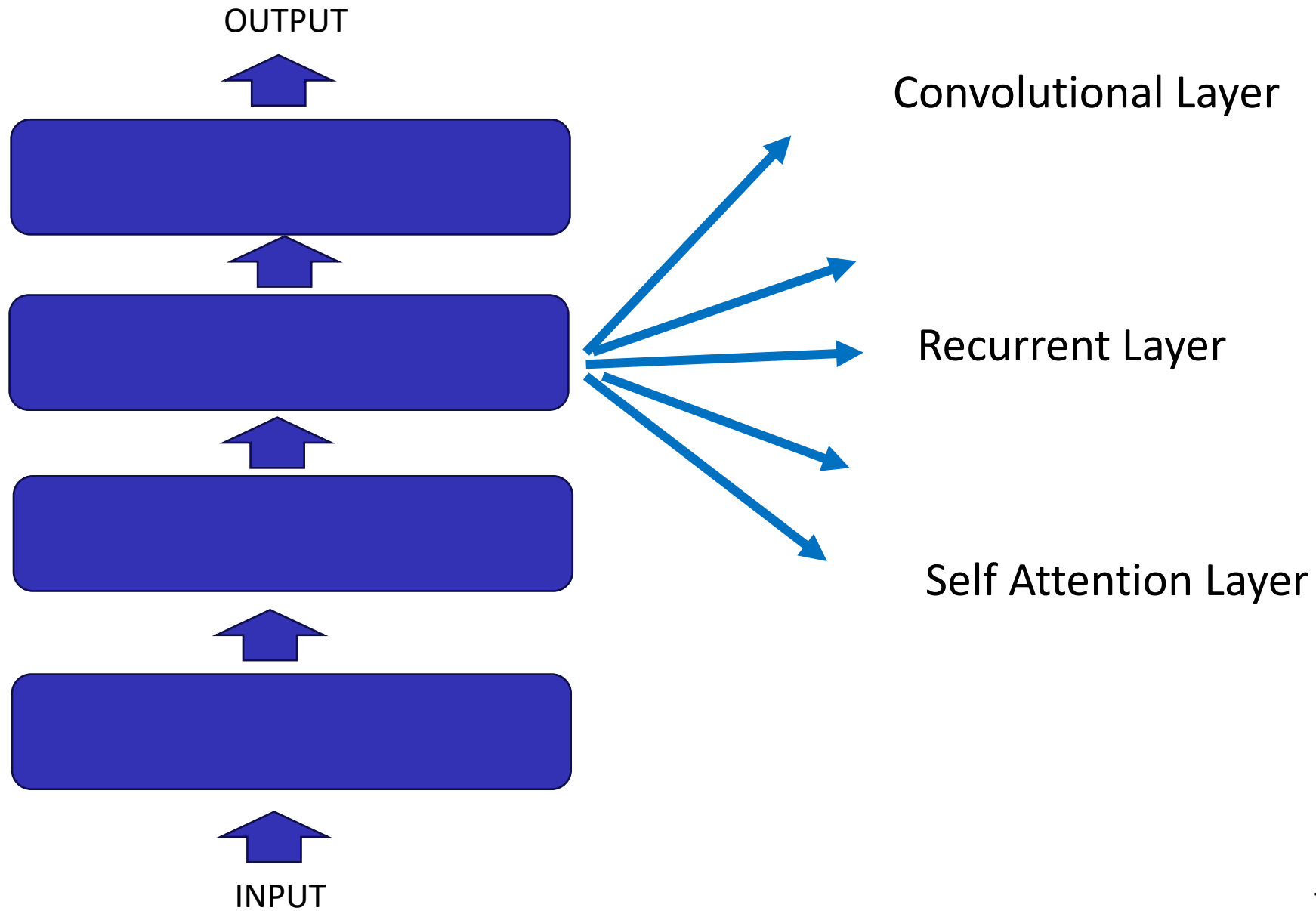
Questions?

Deep Learning

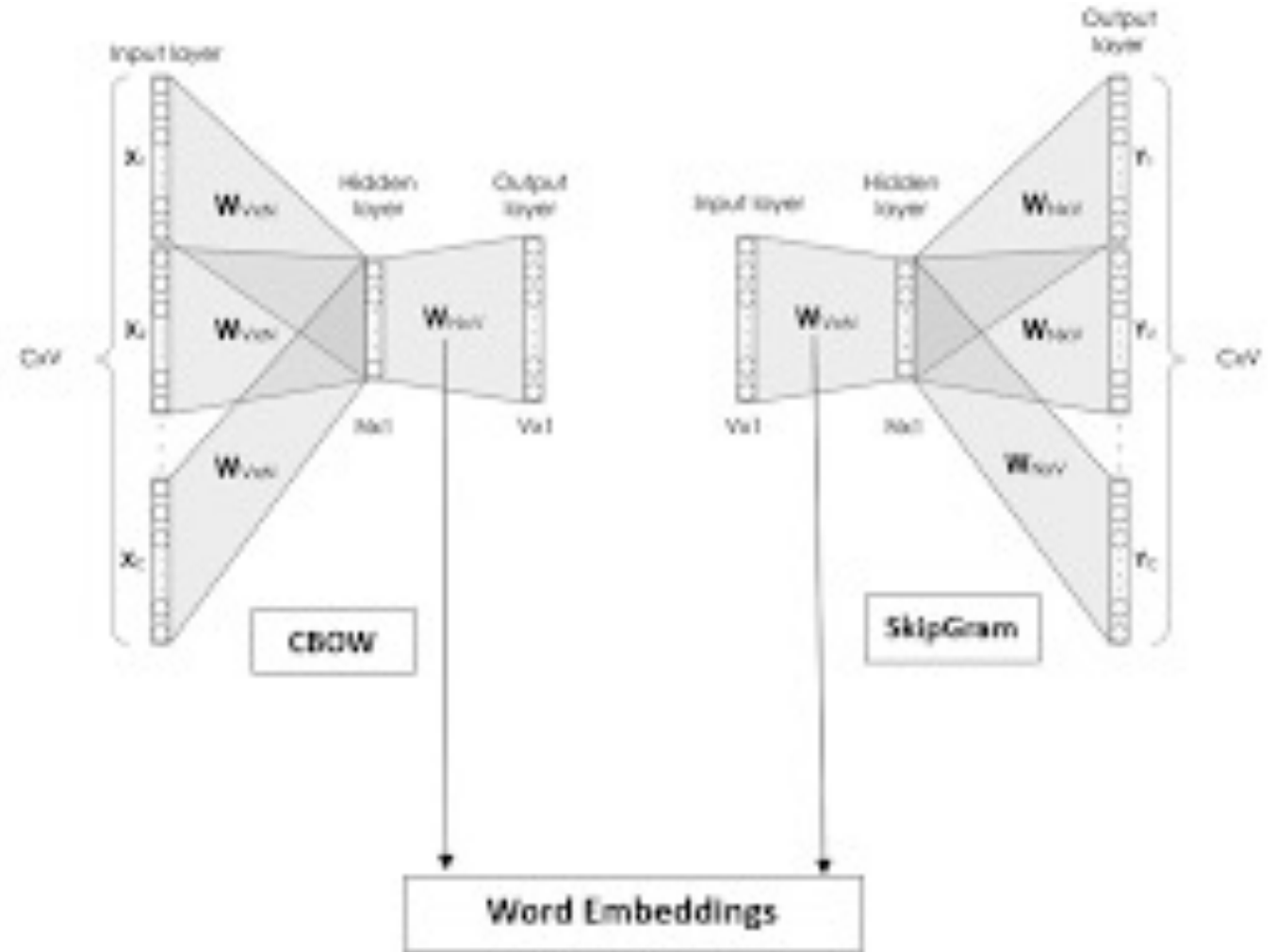
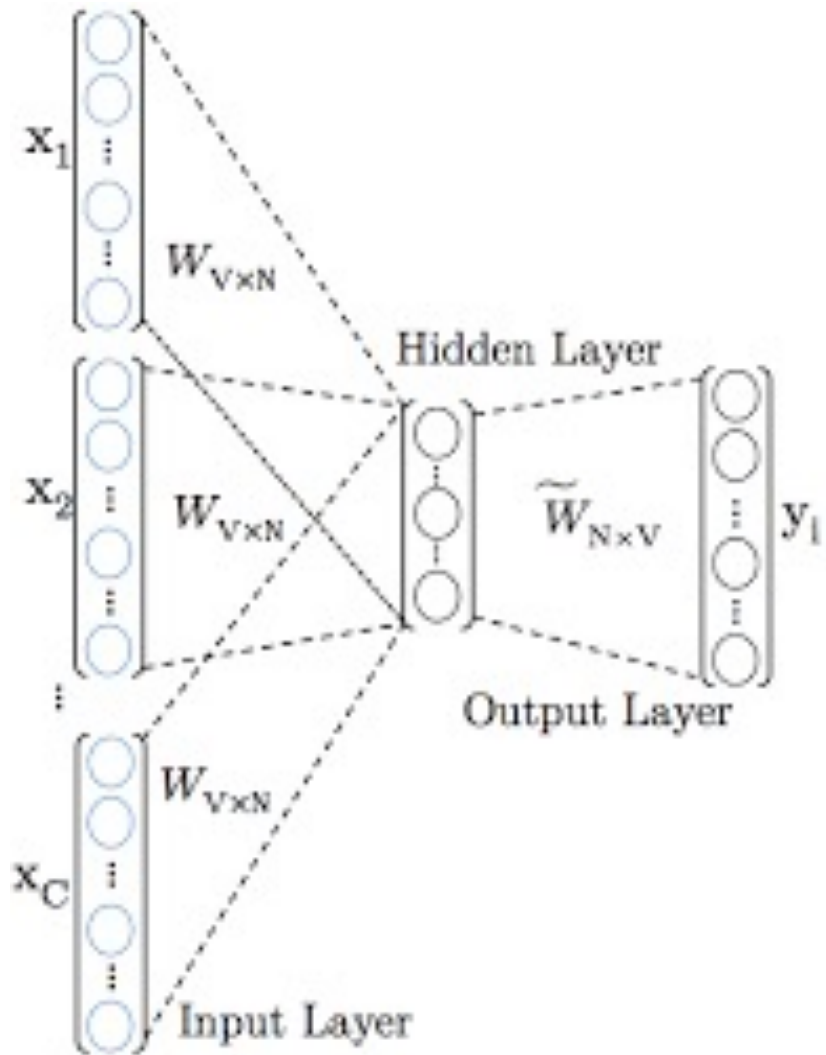
- It's **deep** if it has **more than one stage** of non-linear feature transformation.



DL and Different Types of Layers

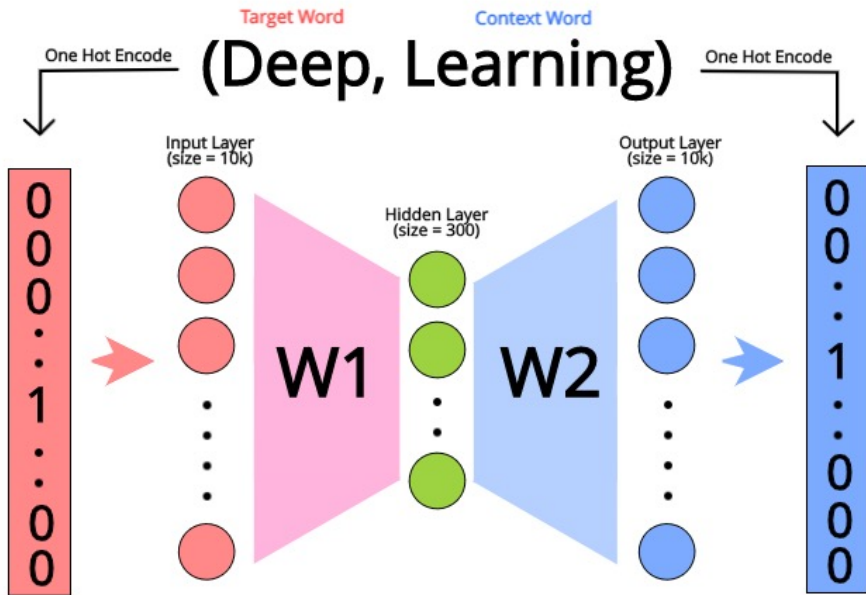


Word Representation

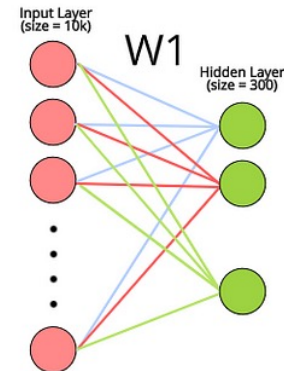
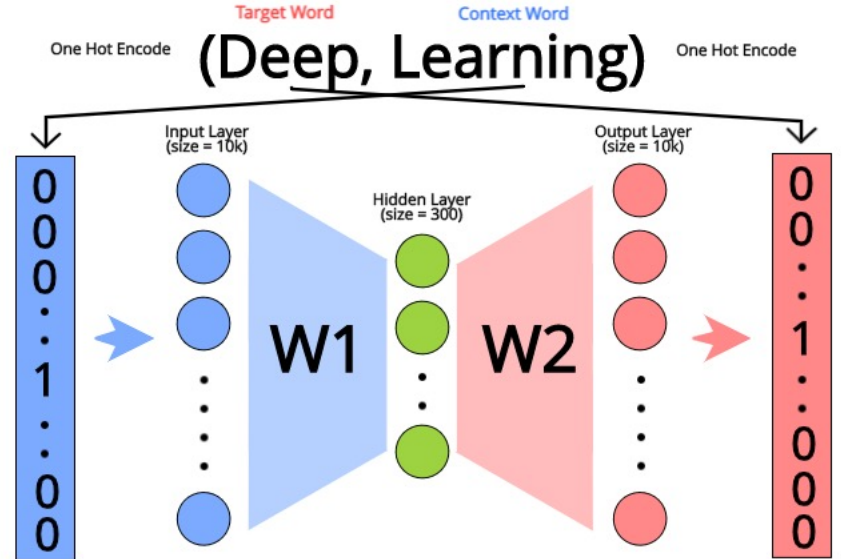


Words to Embeddings: Word2Vec

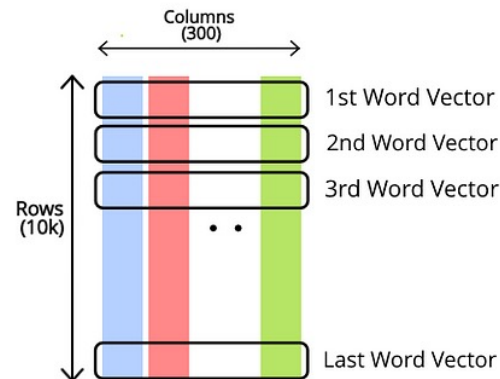
Skip Gram Architecture



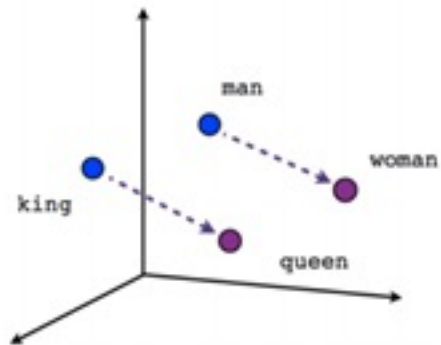
CBOW Architecture



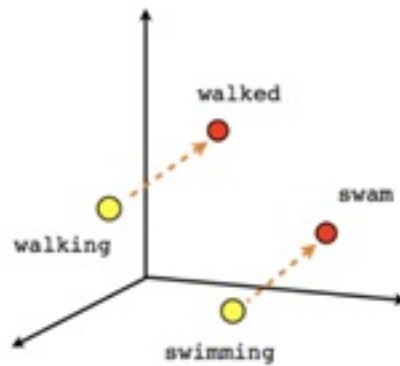
Weight Matrix



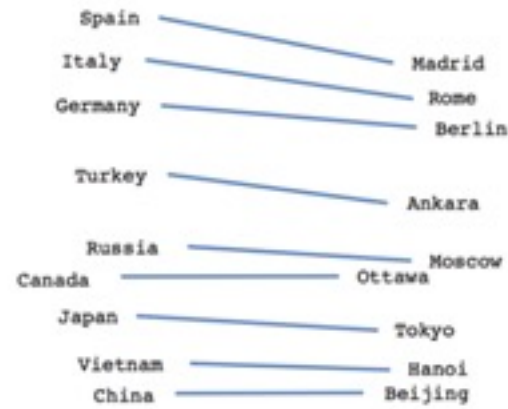
Word Arithmetic



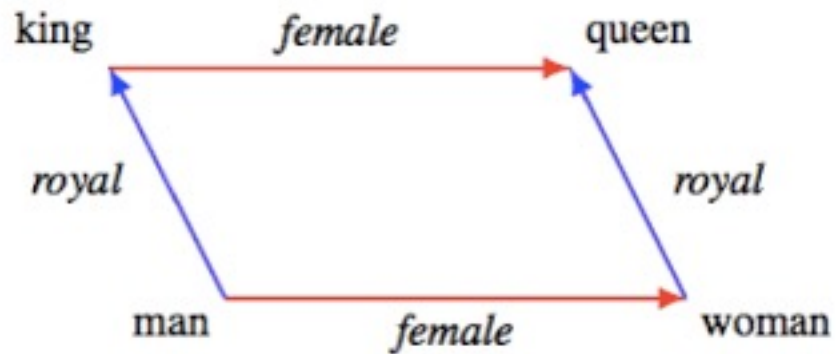
Male-Female



Verb tense



Country-Capital

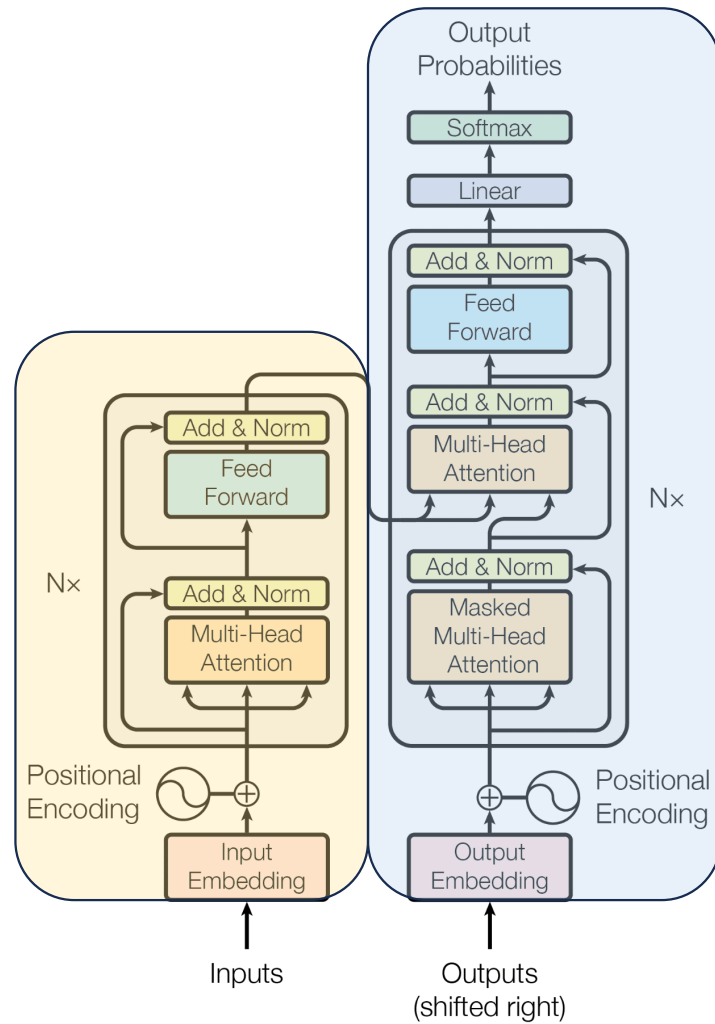


$$\text{King} - \text{Queen} = \text{Man} - \text{Women}$$

Self Attention for Transformers

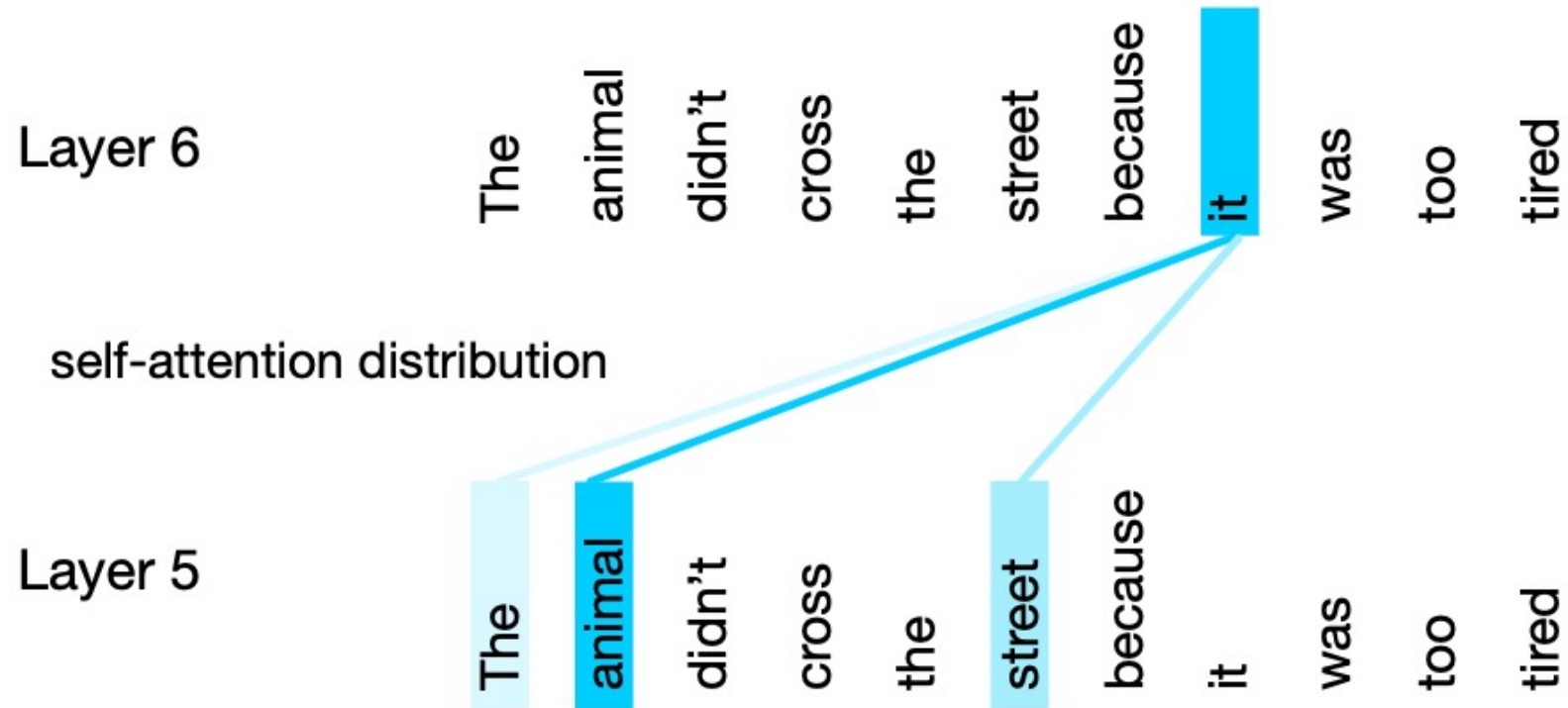
Transformers (2017)

Representation



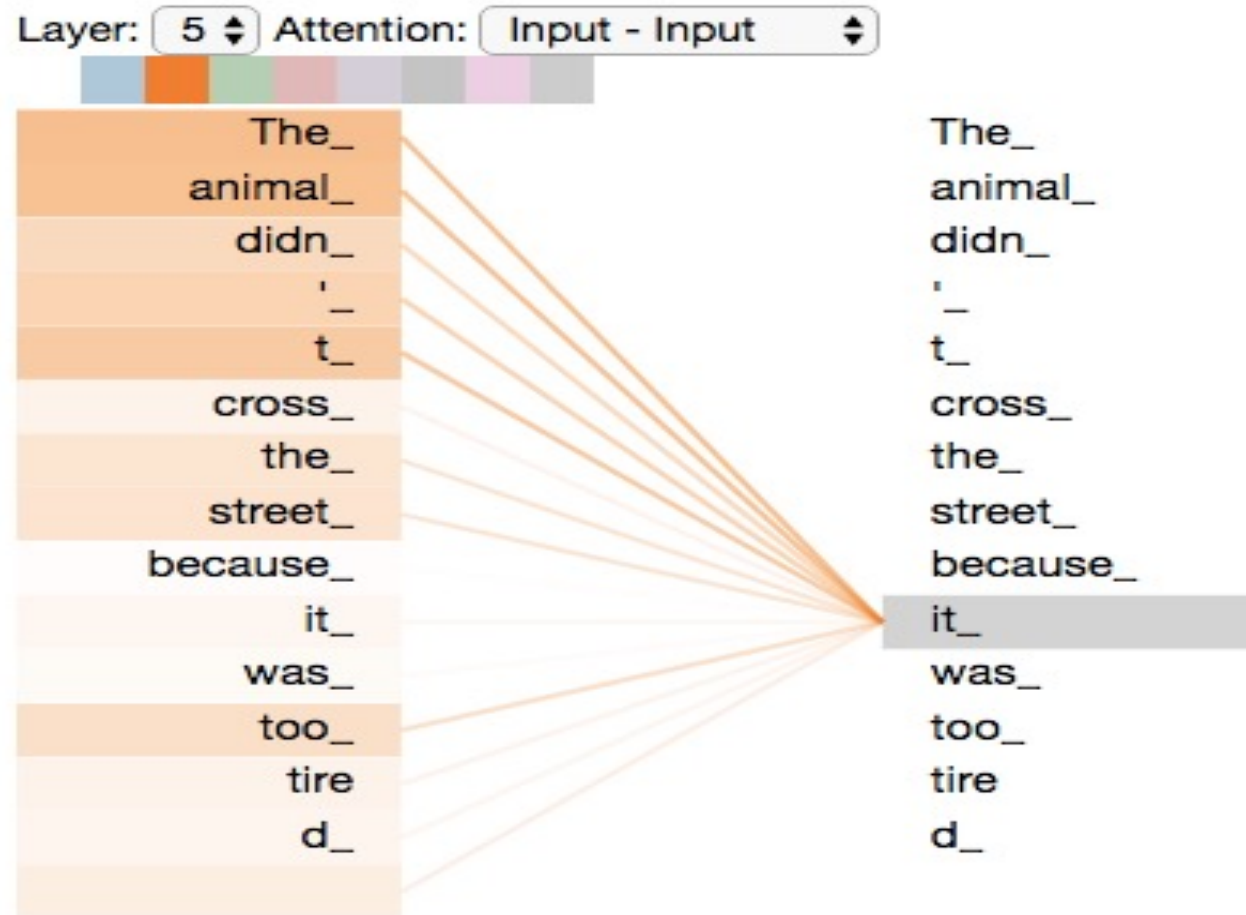
Generation

Self Attention



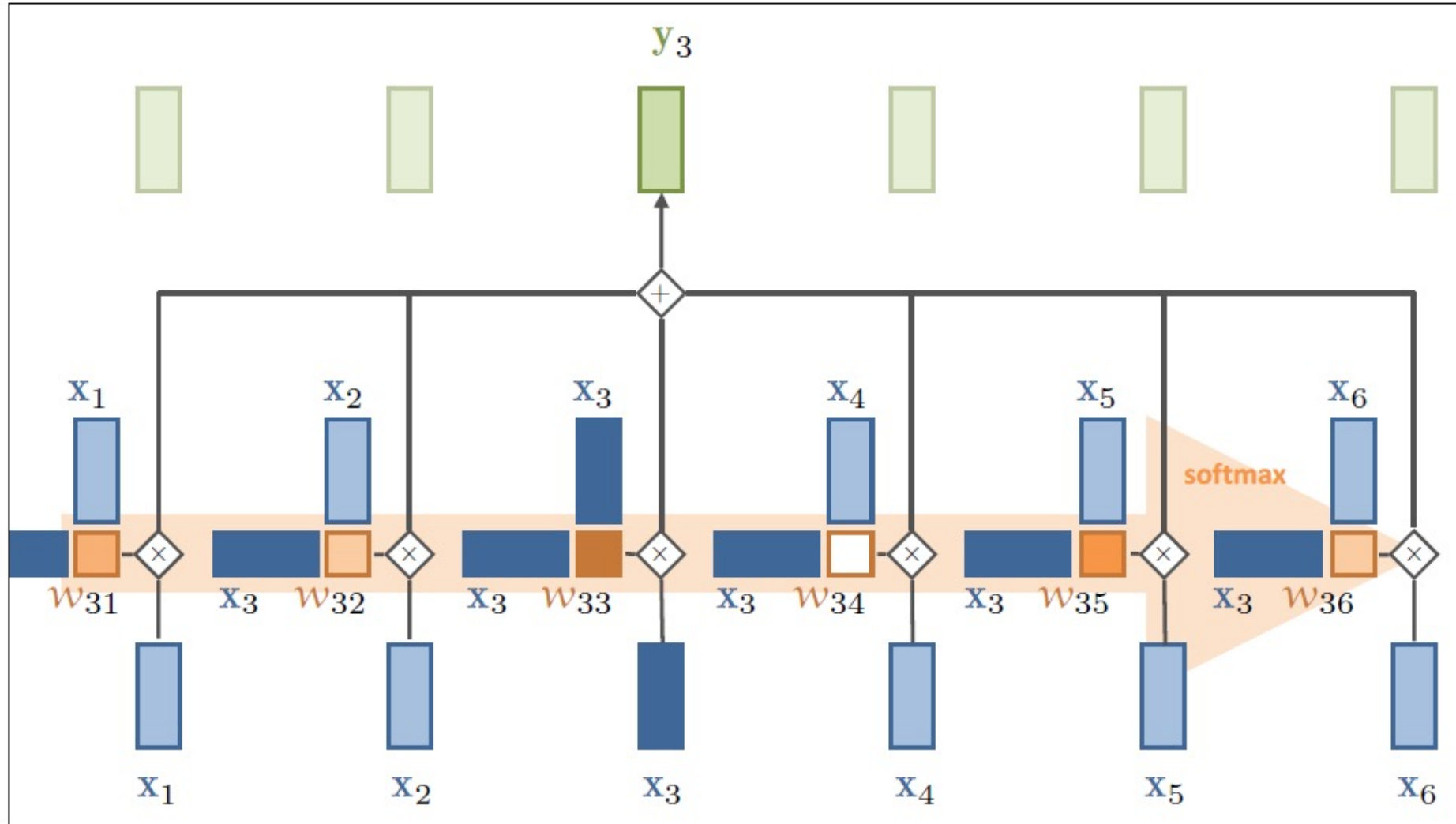
1. **The animal** did not cross the road because **it** was too tired.
2. The animal did not cross **the road** because **it** was too wide.

Self Attention



1. **The animal** did not cross the road because **it** was too tired.
2. The animal did not cross **the road** because **it** was too wide.

SA: Simple Self Attention



$$y_i = \sum_j w_{ij} x_j$$

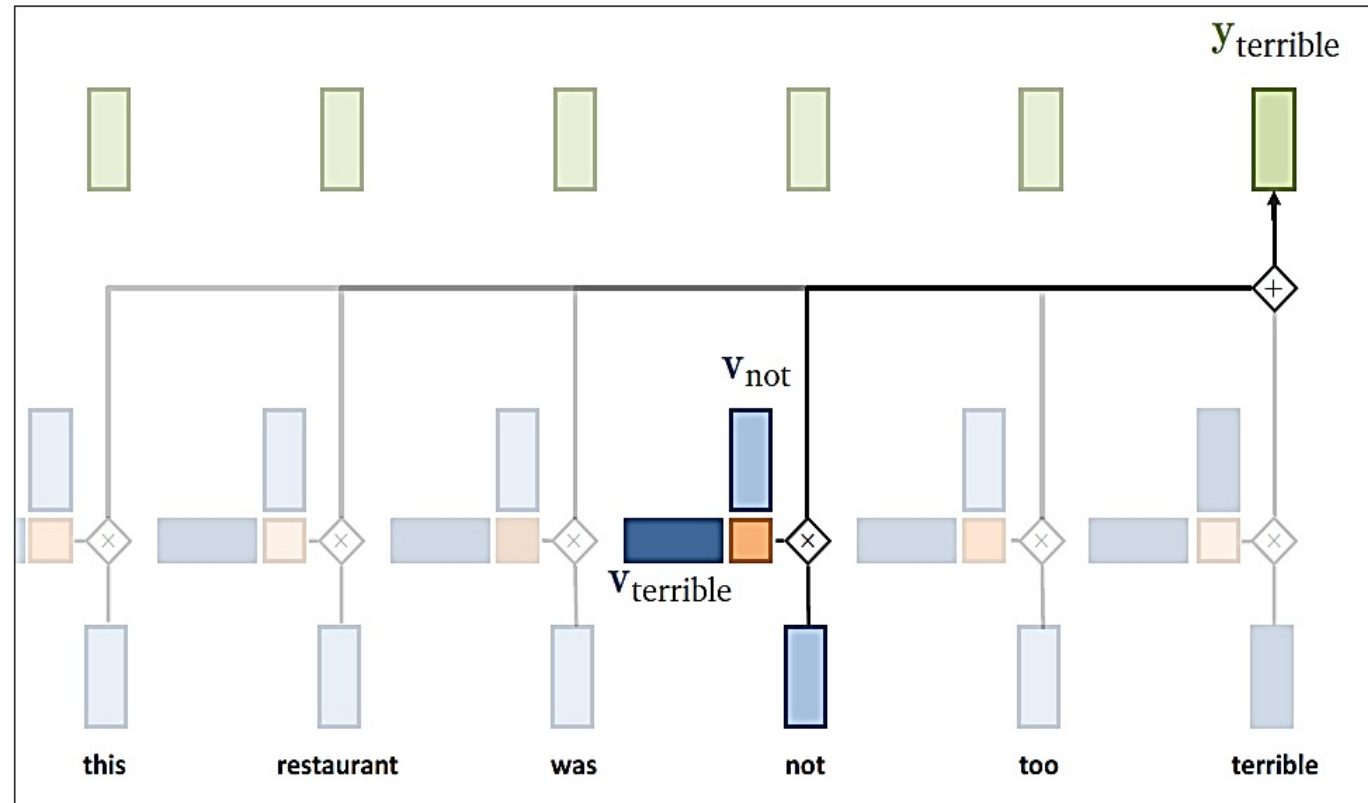
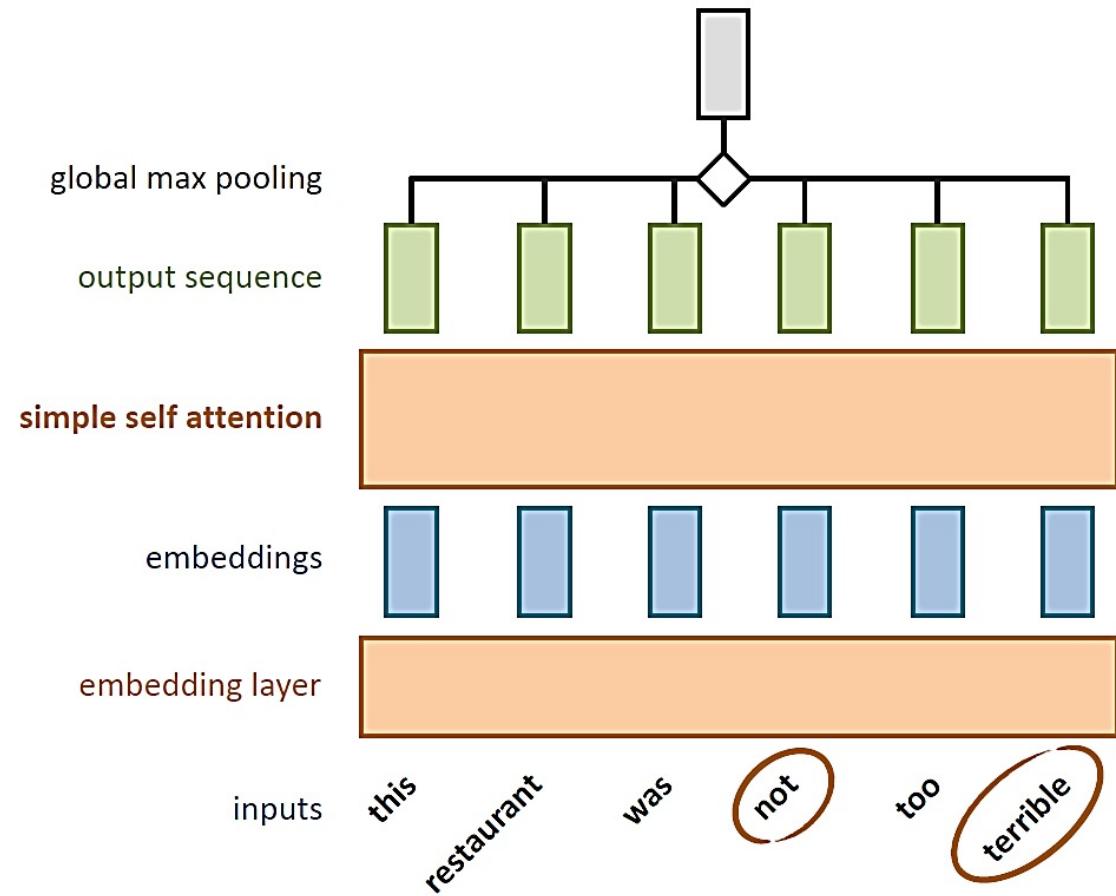
$$w'_{ij} = x_i^T x_j$$

$$w_{ij} = \frac{\exp w'_{ij}}{\sum_l \exp w'_{il}}$$

SA: Observations

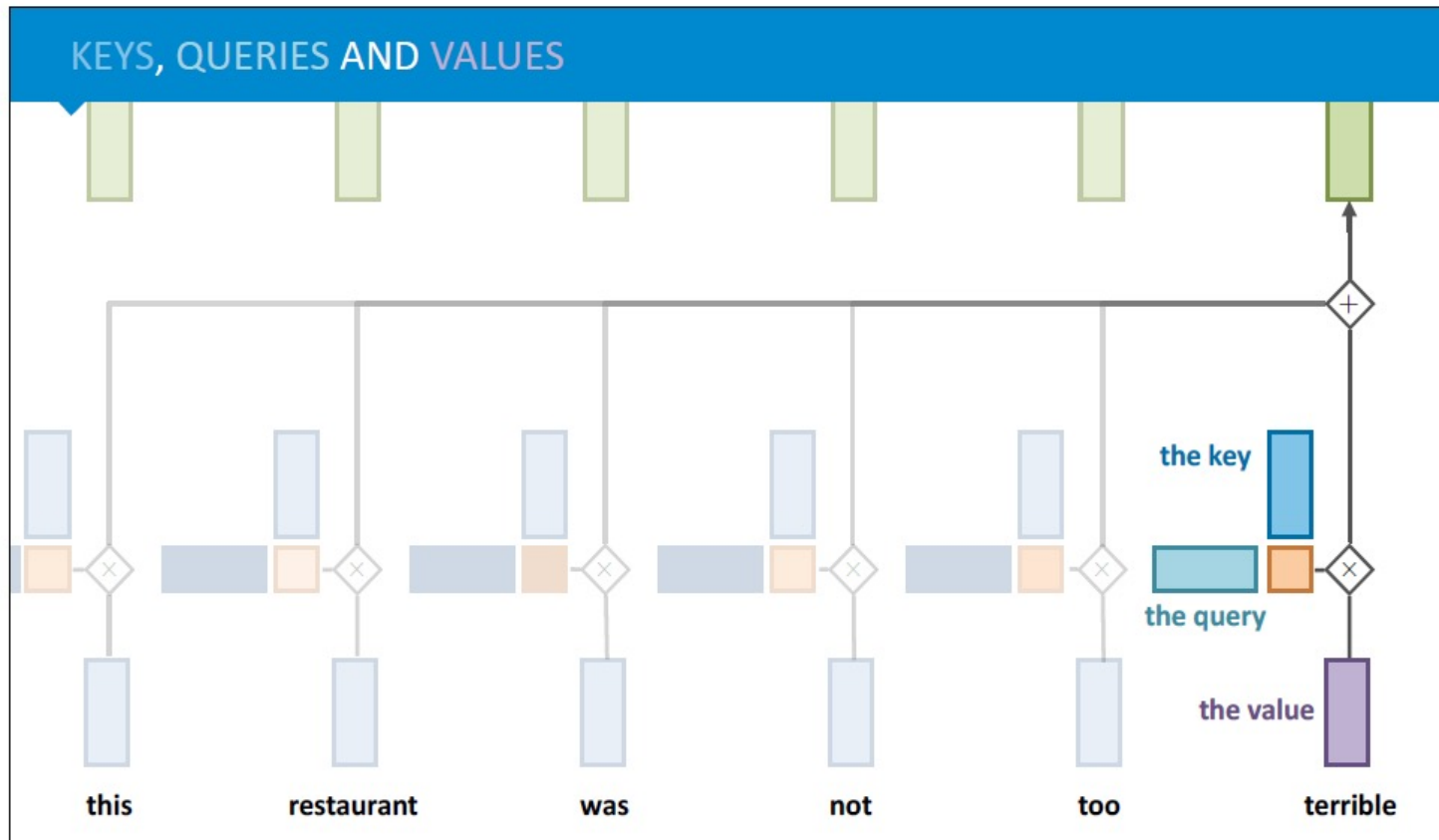
- Input and Output:
 - Vectors are of same in number and dimension
 - Set of vectors (in simple form). Add position, if required.
- Every output vector is influenced by “all” input vectors.
- “ W ” is not learned here. It is computed from the input/data.

Example



Query, Key and Value

KEYS, QUERIES AND VALUES



In each self attention computation, every input vector occurs in three distinct roles:

- **the value**: the vector that is used in the weighted sum that ultimately provides the output
- **the query**: the input vector that corresponds to the current output, matched against every other input vector.
- **the key**: the input vector that the query is matched against to determine the weight.

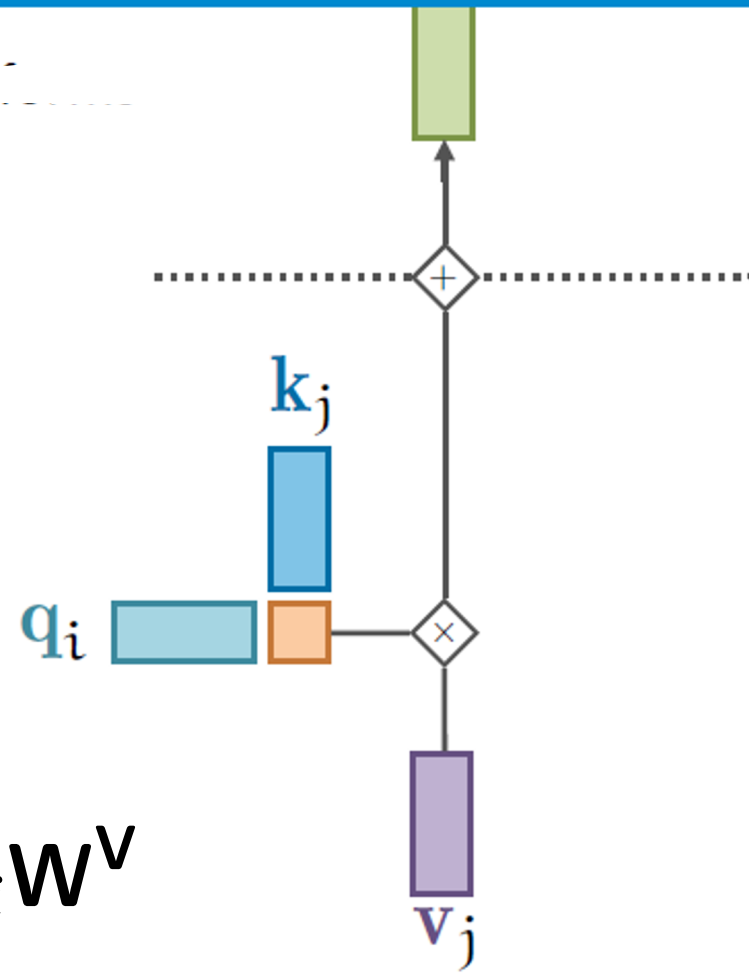
Learnable “Query”, “Key” and “Value”

$$\mathbf{k}_i = \mathbf{K}\mathbf{x}_i + \mathbf{b}_k$$

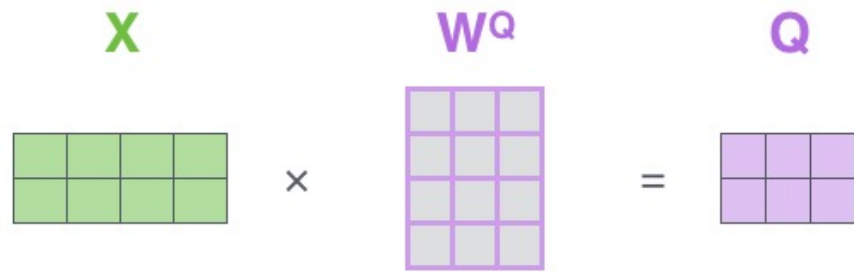
$$\mathbf{q}_i = \mathbf{Q}\mathbf{x}_i + \mathbf{b}_q$$

$$\mathbf{v}_i = \mathbf{V}\mathbf{x}_i + \mathbf{b}_v$$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

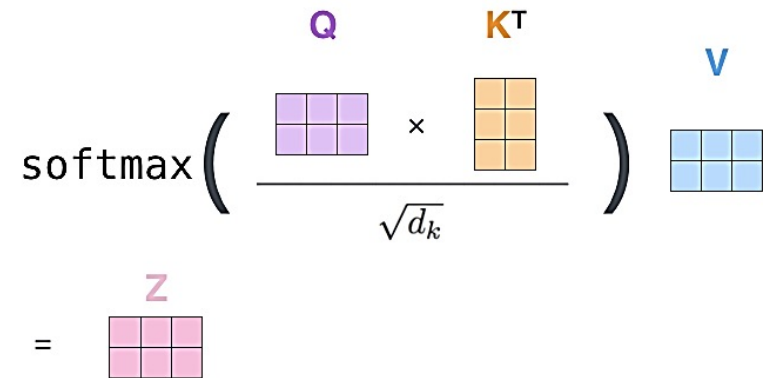


Matrix View

$$\mathbf{X} \times \mathbf{W}^Q = \mathbf{Q}$$


$$\mathbf{X} \times \mathbf{W}^K = \mathbf{K}$$


$$\mathbf{X} \times \mathbf{W}^V = \mathbf{V}$$


$$\text{softmax}\left(\frac{\mathbf{Q} \times \mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$$
$$= \mathbf{Z}$$


Parallelizable Computation

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

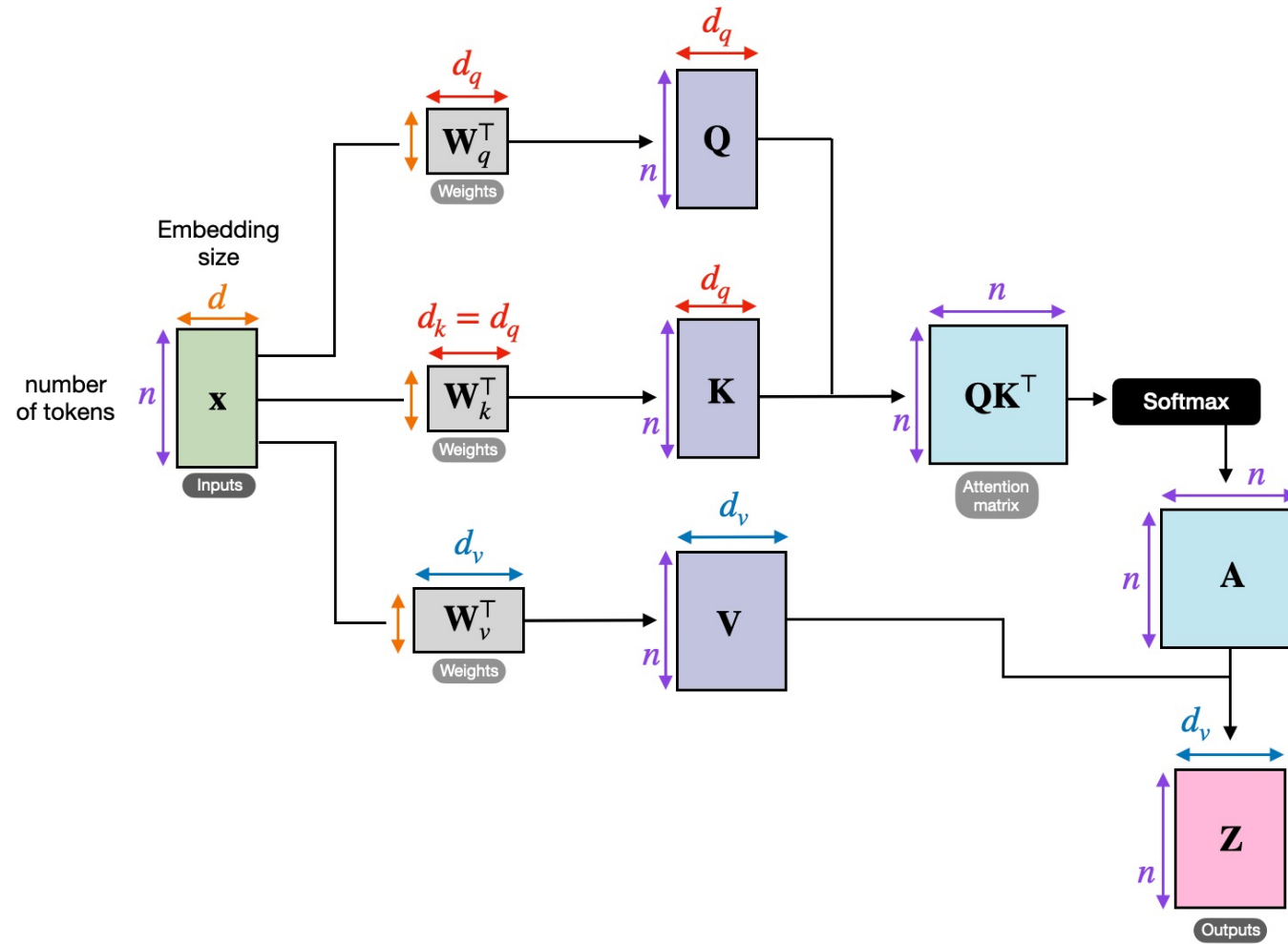
$$\cdot \quad \text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j))$$

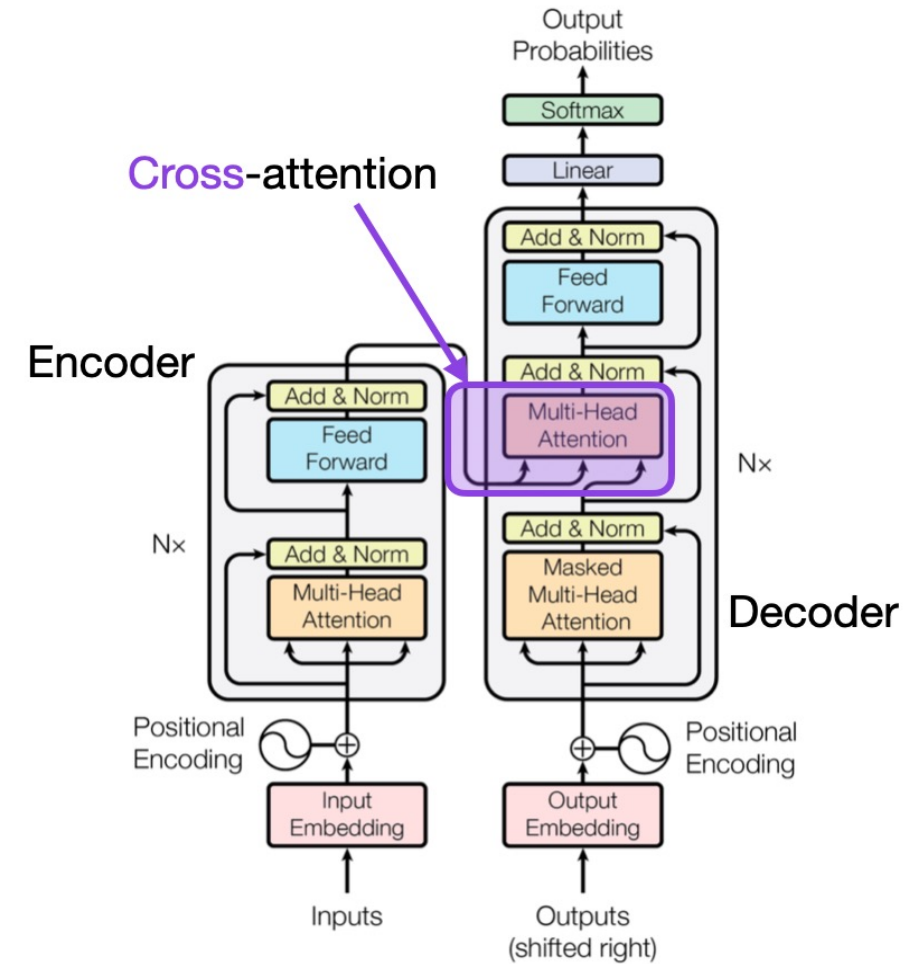
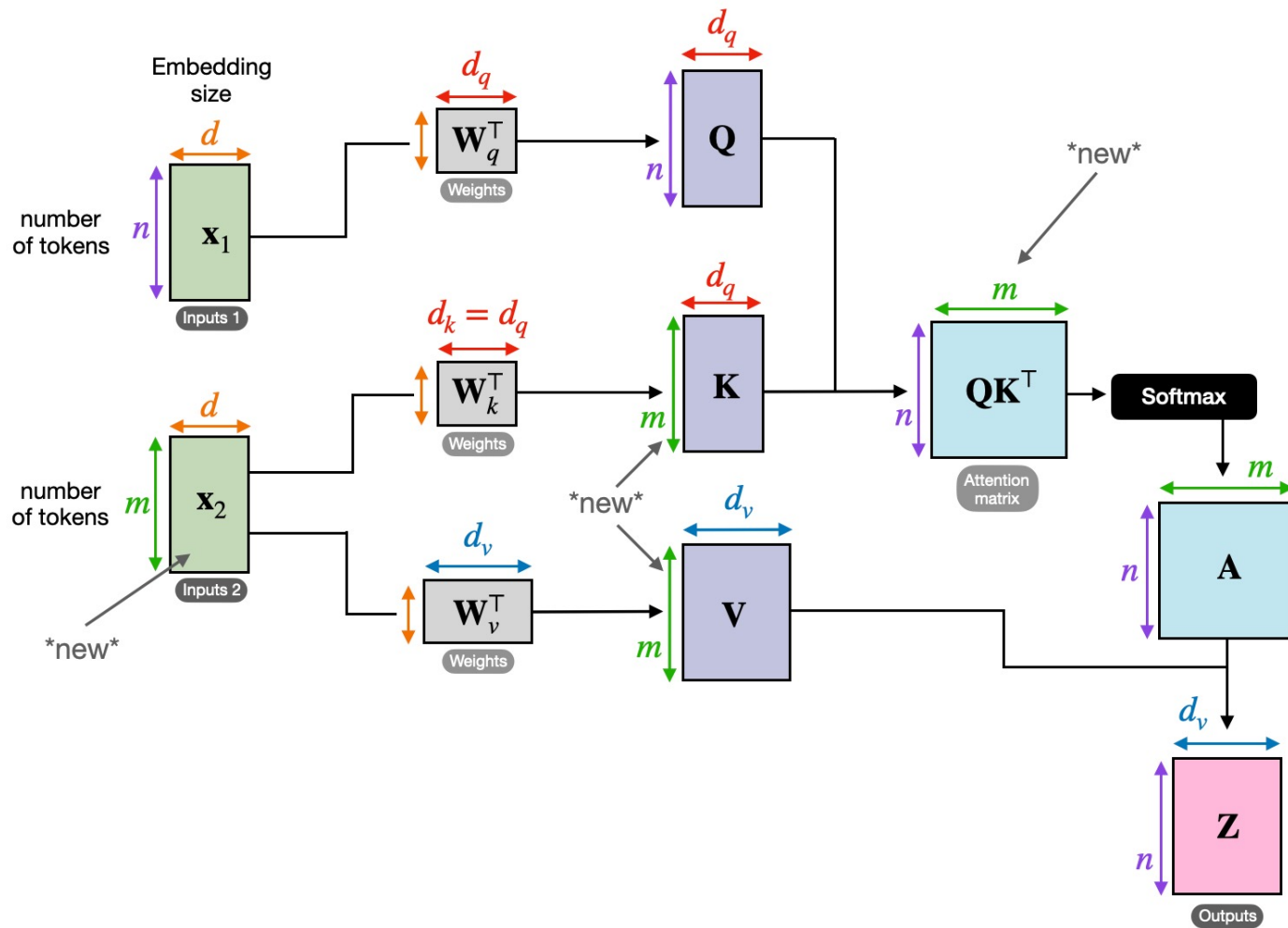
$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

$$\mathbf{Q} = \mathbf{XW}^Q; \mathbf{K} = \mathbf{XW}^K; \mathbf{V} = \mathbf{XW}^V \quad \mathbf{A} = \text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

Self Attention: Computation



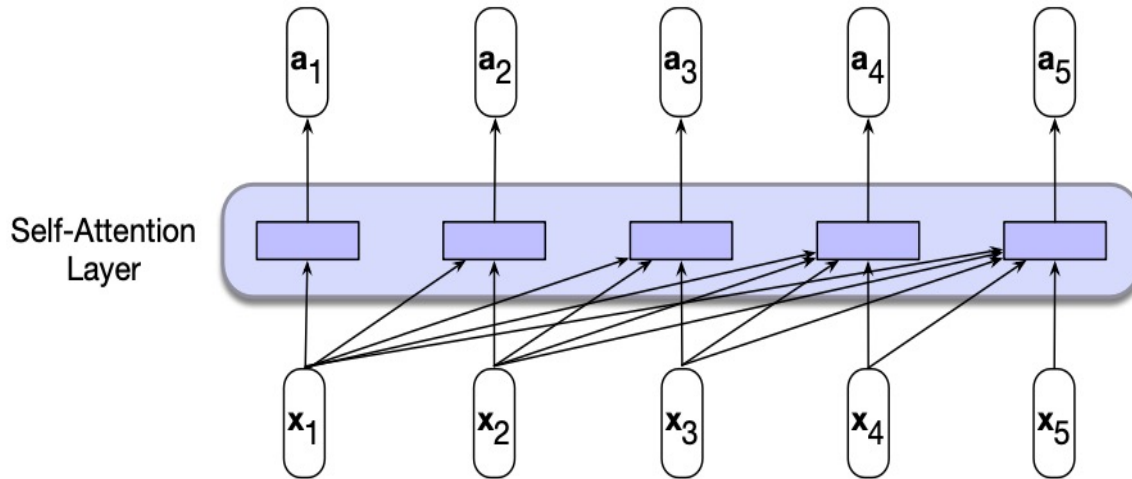
Cross Attention



Source: "Attention Is All You Need" (<https://arxiv.org/abs/1706.03762>)

Example: Note that the queries usually come from the decoder, and the keys and values usually come from the encoder.

Self Attention: Causal/masked



Unlike RNNs, computations at each step are independent of all other time steps and can be done parallelly.

What is the typical value of N?
4096 tokens

Masking out the future

QK^T

N	q1•k1	−∞	−∞	−∞	−∞
	q2•k1	q2•k2	−∞	−∞	−∞
	q3•k1	q3•k2	q3•k3	−∞	−∞
	q4•k1	q4•k2	q4•k3	q4•k4	−∞
	q5•k1	q5•k2	q5•k3	q5•k4	q5•k5
N					

$$attention(Q, K, V) = softmax\left(\frac{Q^T K}{\sqrt{d_k}}\right) V$$

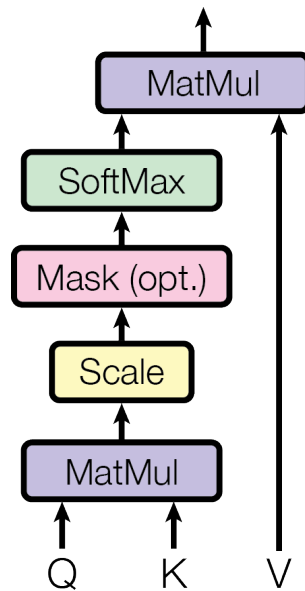
$$maskedAttention(Q, K, V) = softmax\left(\frac{Q^T K + M}{\sqrt{d_k}}\right) V$$

where M is a mask matrix of 0's and $-\infty$'s

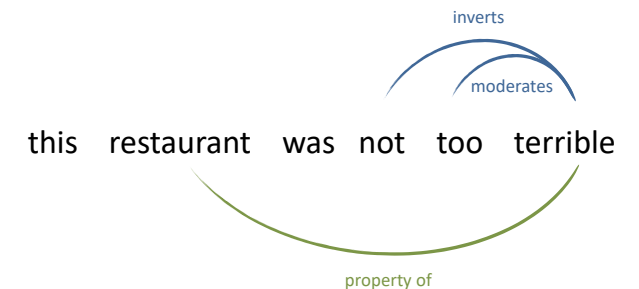
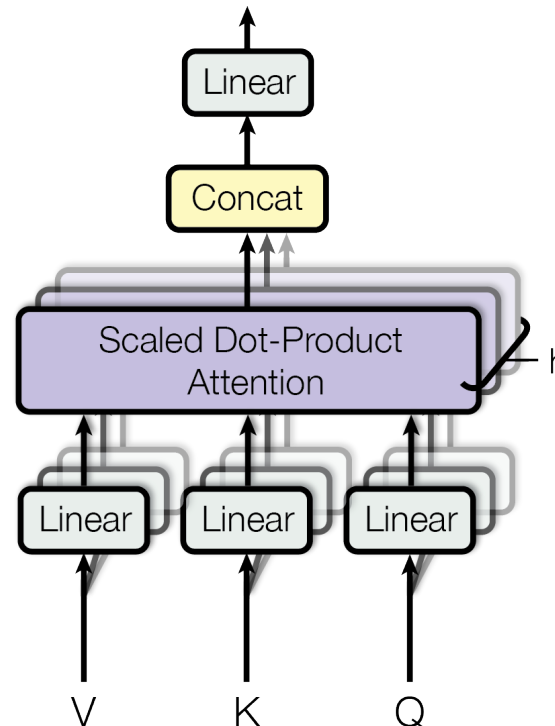
Multi Head Attention: Why?

- Why are we limited to one query, key and value?
- Multiple Feature Maps (like Convolution Layers)

Scaled Dot-Product Attention



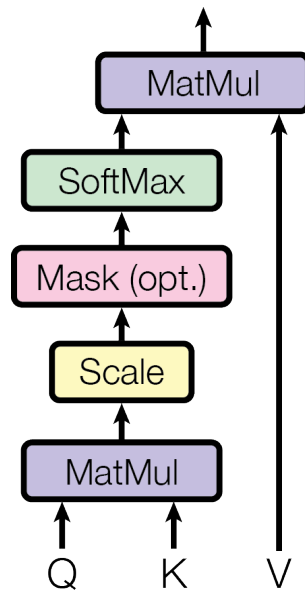
Multi-Head Attention



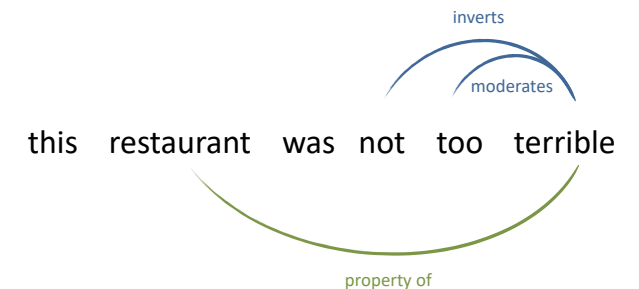
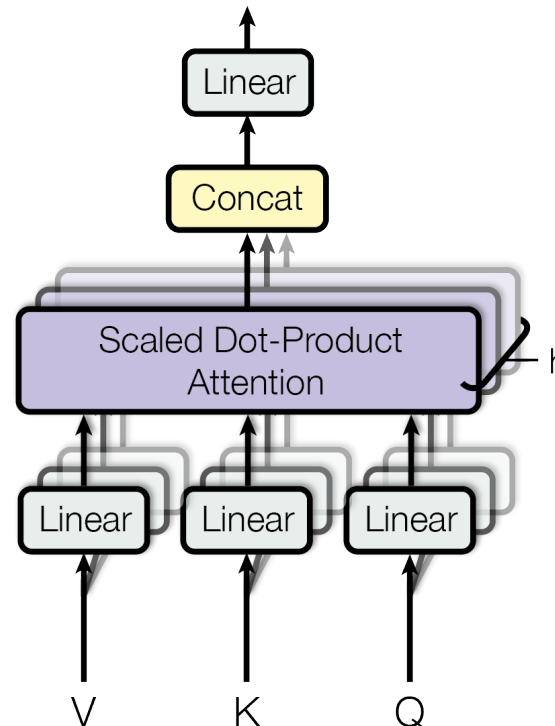
Multi Head Attention: Why?

- Why are we limited to one query, key and value?
- Multiple Feature Maps (like Convolution Layers)

Scaled Dot-Product Attention



Multi-Head Attention

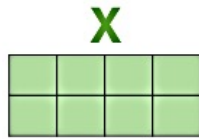


Multi-Head Attention

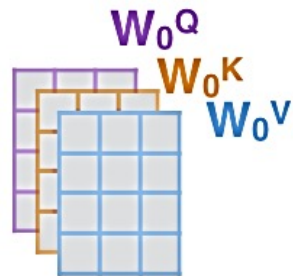
1) This is our input sentence*

Thinking
Machines

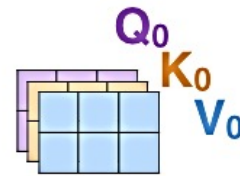
2) We embed each word*



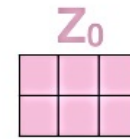
3) Split into 8 heads. We multiply X or R with weight matrices



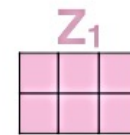
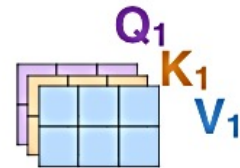
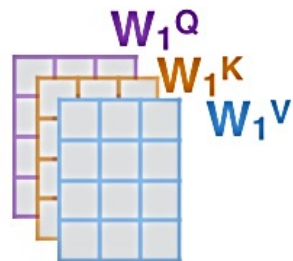
4) Calculate attention using the resulting $Q/K/V$ matrices



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



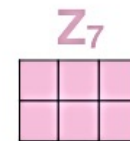
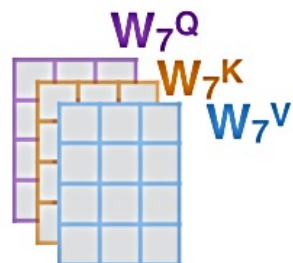
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



...

...

...



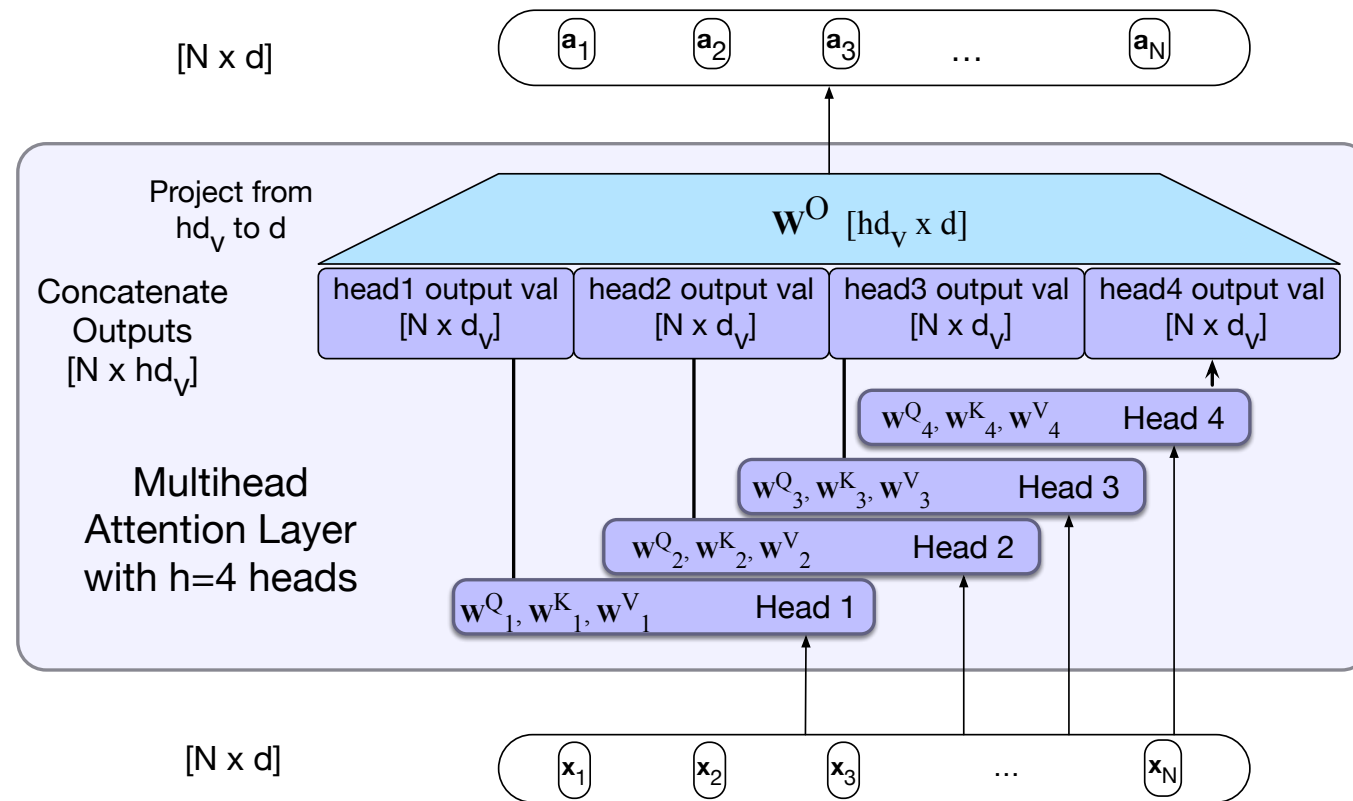
W^O



Z



Multi Head Attention



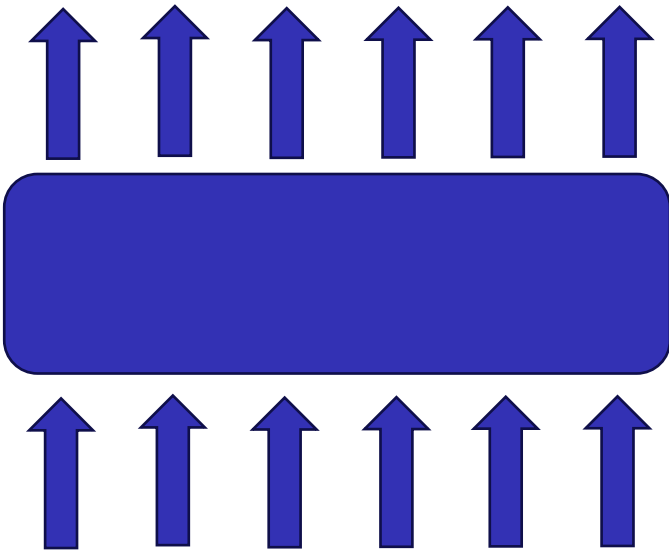
$$\mathbf{Q} = \mathbf{XW}_i^Q ; \mathbf{K} = \mathbf{XW}_i^K ; \mathbf{V} = \mathbf{XW}_i^V$$

$$\text{head}_i = \text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$$

$$\mathbf{A} = \text{MultiHeadAttention}(\mathbf{X}) = (\text{head}_1 \oplus \text{head}_2 \dots \oplus \text{head}_h) \mathbf{W}^O$$

MLP: Fully connected layer

Input: N vectors of d dimension



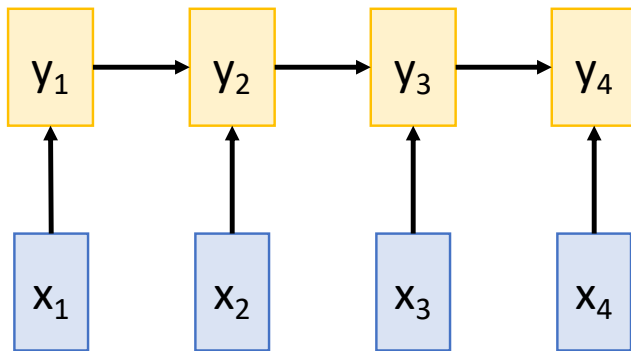
Input: N vectors of d dimension

- Let $N=1000$ and $d = 300$

- MLP
 - Fully Connected
 - Number of parameters to learn
 - $(1000 \times 300) \times (1000 \times 300)$
 - $= 9 \times 10^{10}$
- CNN
 - Let $N=1000$ (say sqr) and $d = 300$
 - Number of parameters to learn
 - $300 \text{ (channels)} \times 3 \times 3 \times 300$
 - $= 27 \times 10^4$
- RNN
 - Let $N=1000$ and $d = 300$ (and $h = 100$)
 - Number of parameters to learn
 - $300 \times 100 \text{ (U)}; 100 \times 100 \text{ (W)}; 100 \times 300 \text{ (V)}$
 - $= 7 \times 10^4$
- SA
 - Let $N=1000$ and $d = 300$; Project to h ; $h = 100$
 - $300 \times 100 \text{ (W}^Q\text{)}; 300 \times 100 \text{ (W}^K\text{)}; 300 \times 100 \text{ (W}^V\text{)}; 100 \times 300 \text{ (W}^O\text{)}$
 - $= 12 \times 10^4$

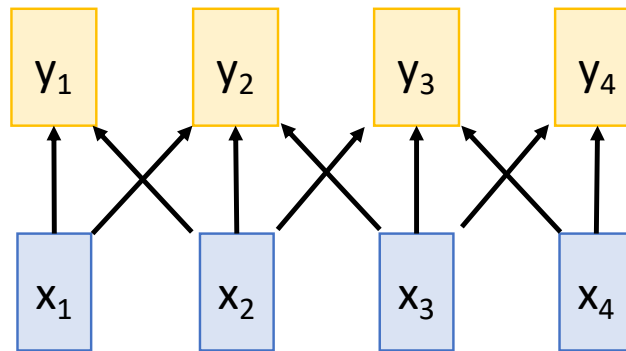
Comparison

Recurrent Neural Network



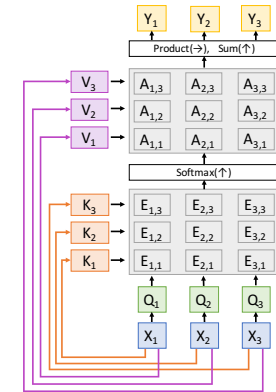
- Works on Ordered Sequences
- Long range dependence (in the past)
- Not parallelizable

1D Convolution



- Works on Multidimensional grids
- Short visibility; Stacking helps
- Highly parallelizable

Self-Attention

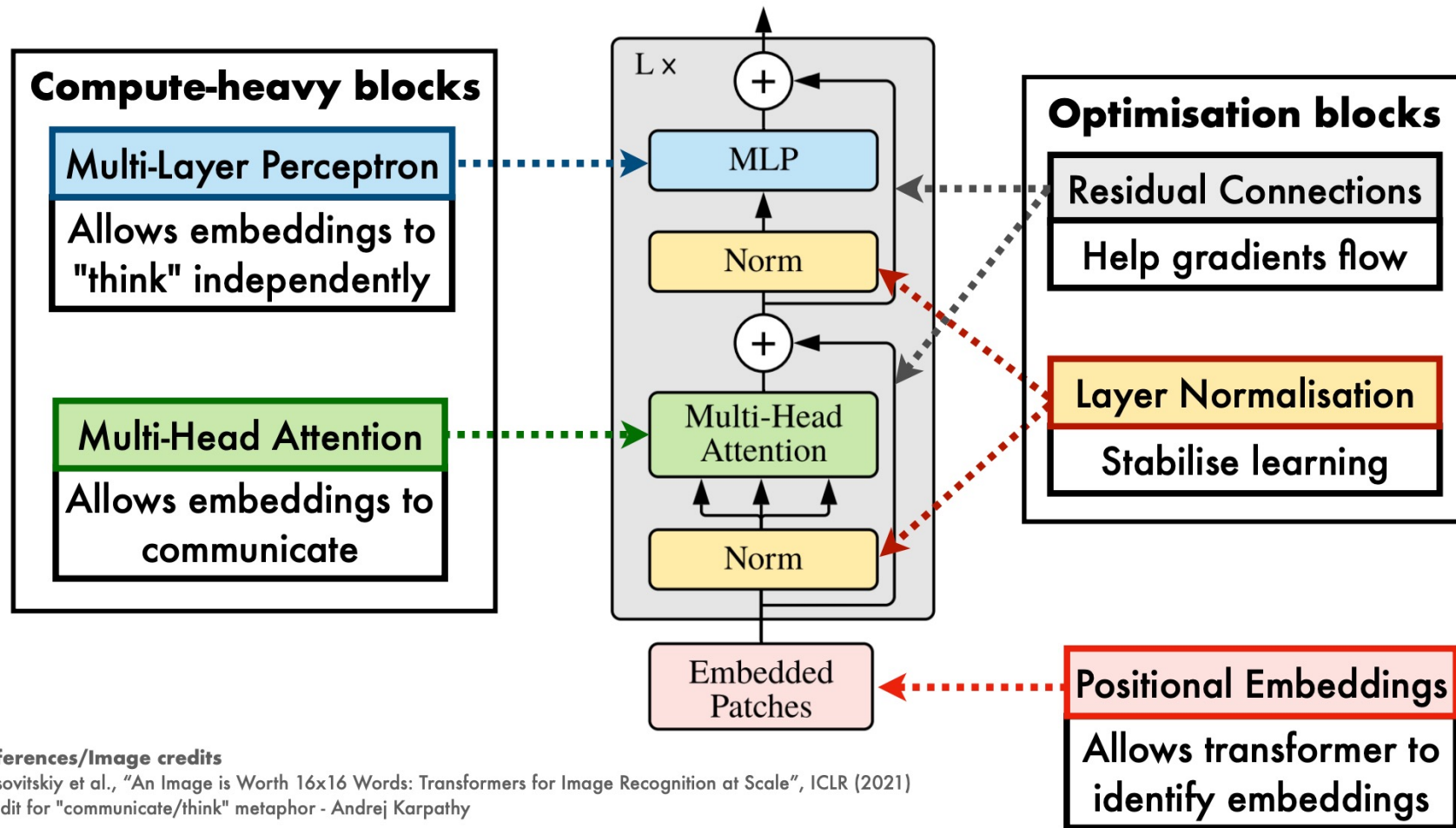


- Works on Sets and Sequences
- Long range dependence
- Parallelizable
- Memory Intensive

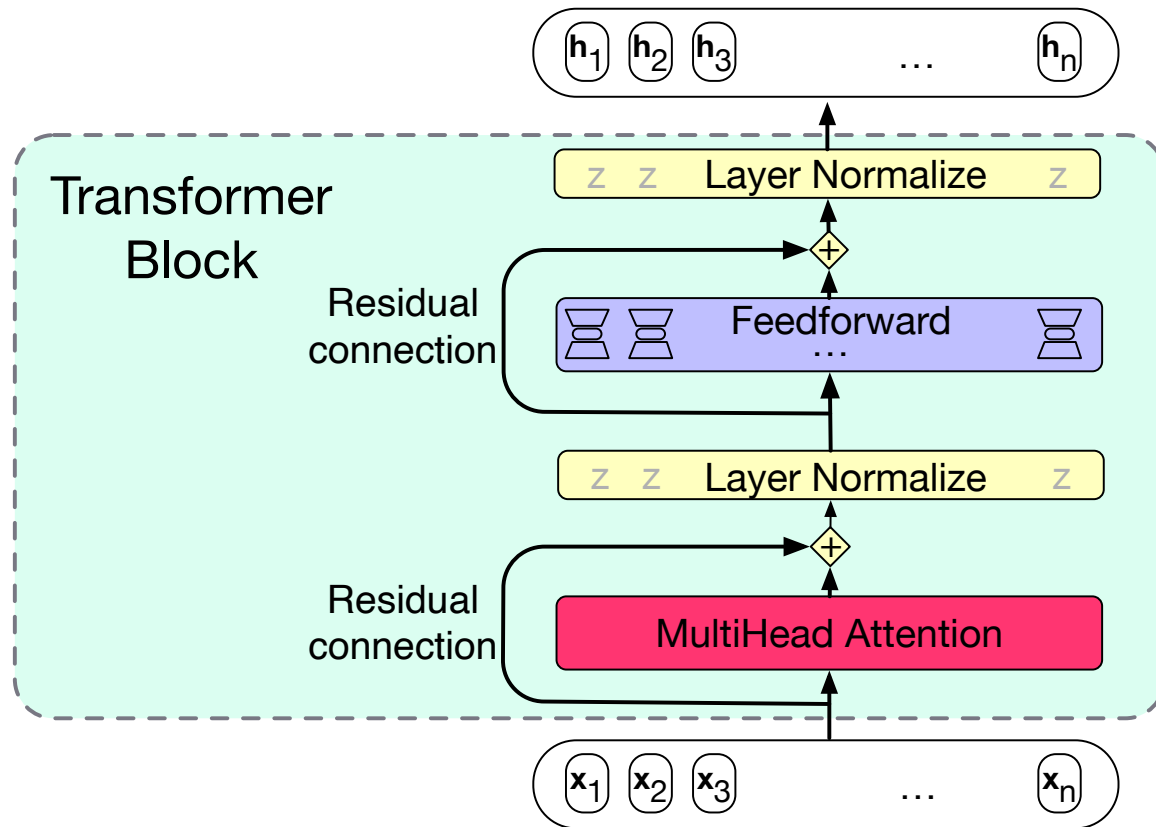
Transformers

Transformer Encoder

Five key ideas



Transformer Block



Both X and H are of $N \times d$

$$\mathbf{T}^1 = \text{SelfAttention}(\mathbf{X})$$

$$\mathbf{T}^2 = \mathbf{X} + \mathbf{T}^1$$

$$\mathbf{T}_3 = \text{LayerNorm}(\mathbf{T}^2)$$

$$\mathbf{T}^4 = \text{FFN}(\mathbf{T}^3)$$

$$\mathbf{T}^5 = \mathbf{T}^4 + \mathbf{T}^3$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{T}^5)$$

Questions?